

AWS CDK

Basics & Core Terminology

A Practitioner's Handbook — Every Term Explained with Examples

Chapter 1 · What is AWS CDK?

Chapter 2 · Core Terminologies Explained

Chapter 3 · Constructs — L1, L2, L3 Deep-Dive

Chapter 4 · Stacks, Apps & Environments

Chapter 5 · The CDK Toolkit & CLI Commands

Chapter 6 · Assets, Bundling & Dependencies

Chapter 7 · CDK Workflows & Lifecycle

Chapter 1

What is AWS CDK?

The **AWS Cloud Development Kit (CDK)** is an open-source framework that lets developers define AWS cloud infrastructure using real programming languages — TypeScript, Python, Java, C#, or Go — instead of writing verbose YAML or JSON templates. CDK was officially released in stable form in 2019 and has since become one of the most popular tools for Infrastructure as Code (IaC) on AWS.

Under the hood, CDK does not invent a new deployment engine. Instead, it **synthesizes** your code into AWS CloudFormation templates, which are then used to create and manage real AWS resources. This means CDK inherits all the reliability, rollback capabilities, and resource management features of CloudFormation, while dramatically improving the developer experience.

Think of CDK as a powerful abstraction layer: you write expressive, readable code — complete with loops, functions, conditionals, and classes — and CDK translates it into the exact CloudFormation resources AWS needs. This bridges the gap between the world of software development and the world of cloud infrastructure management.

- AWS CDK lets you model cloud infrastructure with the same programming languages, tools, and practices used for application code — bringing software engineering discipline (testing, code review, versioning, reuse) to cloud infrastructure.

The Infrastructure as Code (IaC) Landscape

Before CDK, teams used several different approaches to manage cloud infrastructure:

Tool	Language	Abstraction	Cloud Support	Key Weakness
AWS CloudFormation	YAML / JSON	Low	AWS only	Verbose; no real logic
AWS CDK	TS/Python/Java/C#/Go	High	AWS (CDKtf for multi)	AWS-only native
Terraform	HCL	Medium	Multi-cloud	New DSL to learn
Pulumi	TS/Python/Go/.NET	High	Multi-cloud	Smaller community
AWS SAM	YAML	Medium	AWS serverless only	Limited scope

CDK's Core Promise

- **Developer-native:** Use your existing language, IDE, and testing tools — no new DSL.
- **Abstraction at scale:** Go from raw CloudFormation resources (L1) to entire architectural patterns (L3) with a single construct.
- **Reusability:** Package infrastructure patterns as libraries and share across teams via npm or PyPI.
- **Safety:** Compile-time type checking catches misconfigured resources before any AWS API call is made.

- **Built on CloudFormation:** Full change-set previews, drift detection, and rollback — for free.

Chapter 2

Core Terminologies Explained

CDK introduces a well-defined vocabulary that every practitioner must understand. Each term represents a specific concept in the CDK mental model. Below, every term is explained in depth with practical context and examples.

Construct

A **Construct** is the fundamental building block of every CDK application. It represents a cloud component — anything from a single AWS resource (like an S3 bucket) to a complex multi-resource pattern (like a fully configured ECS Fargate service with a load balancer, VPC, security groups, IAM roles, and CloudWatch alarms). Every element you create in CDK is a Construct. Constructs are composable: you can nest constructs inside other constructs, building sophisticated abstractions from simpler pieces. All Constructs take three arguments: **scope** (the parent construct), **id** (a unique identifier within the parent), and **props** (a configuration object).

Example: `new s3.Bucket(this, 'MyBucket', { versioned: true })` — creates an S3 bucket Construct.

Note: Every CDK class — Stack, App, and all resource classes — is a Construct.

App

The **App** is the root Construct — the top-level container for your entire CDK program. Every CDK application starts by instantiating an App. The App's primary responsibility is to act as the synthesis target: when you run `cdk synth`, the App collects all Stacks registered with it and synthesizes each one into a CloudFormation template. The App itself does not represent any AWS resource — it is purely an organizational and synthesis orchestrator. You can register multiple Stacks to a single App, enabling multi-account, multi-region deployments from one codebase.

Example: `const app = new cdk.App();` — creates the App that owns all stacks.

Note: Always call `app.synth()` at the end of your entry file, or CDK will not generate outputs.

Stack

A **Stack** is the unit of deployment in CDK. Each Stack maps 1:1 to a single AWS CloudFormation Stack. When you deploy, CDK sends the synthesized CloudFormation template for each Stack to AWS, which then creates or updates the resources defined in it. Stacks are scoped to a specific AWS Account and Region. A Stack is also a Construct — it sits between the App (its parent) and the resource Constructs (its children). Multiple Stacks can have explicit dependencies (`stack.addDependency()`), ensuring they are deployed in the correct order.

Example: `class NetworkStack extends Stack { }` — defines a Stack for all networking resources.

Note: CloudFormation limits a single Stack to 500 resources. Use Nested Stacks or split into multiple Stacks for large applications.

Environment (env)

An **Environment** in CDK is a combination of an **AWS Account ID** and an **AWS Region**. Every Stack is deployed into a specific Environment. You can either hardcode the environment values (`account='123456789'`, `region='us-east-1'`) for predictable deployments, or use environment variables (`process.env.CDK_DEFAULT_ACCOUNT`) to make Stacks portable across accounts. Environment-aware Stacks can reference environment-specific resources (like existing VPCs or certificate ARNs). Environment-agnostic Stacks are more portable but cannot reference account-specific resources at synthesis time.

Example: `env: { account: '111122223333', region: 'ap-south-1' }`

Note: If you omit the environment, CDK creates an 'environment-agnostic' stack. Some features (like VPC lookups) require an explicit environment.

Chapter 3

Constructs Deep-Dive — L1, L2 & L3 Levels

Constructs exist on a spectrum of abstraction, organized into three distinct levels. Understanding which level to use — and when — is one of the most important skills for any CDK practitioner. Higher levels offer more convenience and built-in best practices; lower levels offer maximum control.

L1 Constructs — CloudFormation Resources (Cfn*)

Level 1 (L1) constructs are the lowest level of abstraction in CDK. They are auto-generated directly from the CloudFormation resource specification and map 1:1 to every CloudFormation resource type. Their class names always start with **Cfn** (e.g., `CfnBucket`, `CfnFunction`, `CfnSecurityGroup`). Using L1 constructs, you have complete and exact control over every property of the CloudFormation resource — nothing is hidden or defaulted. L1s are the right choice when: (a) an L2 for the resource doesn't exist yet, (b) you need to set a property that the L2 doesn't expose, or (c) you want absolute control over the generated CloudFormation.

Example: `new s3.CfnBucket(this, 'RawBucket', { bucketName: 'my-raw-bucket', versioningConfiguration: { status: 'Enabled' } })`

Note: L1 is verbose — you must set every required property yourself, including security configurations that L2 handles automatically.

L2 Constructs — AWS Constructs (Curated & Opinionated)

Level 2 (L2) constructs are the most commonly used level. They are hand-crafted by the CDK team and provide a rich, developer-friendly API on top of the underlying CloudFormation resources. L2 constructs offer: **(1) Sensible defaults** — encryption, versioning, and secure settings are often enabled by default; **(2) Helper methods** — methods like `bucket.grantRead(lambda)` automatically create the minimal required IAM policy; **(3) Event integration** — easy wiring between services (e.g., S3 triggers Lambda on object creation). L2 constructs make it very hard to accidentally create insecure or misconfigured infrastructure.

Example: `const bucket = new s3.Bucket(this, 'Bucket', { encryption: BucketEncryption.S3_MANAGED, blockPublicAccess: BlockPublicAccess.BLOCK_ALL });`

Note: L2 is the sweet spot for most use cases. It balances control with convenience, and you can always 'escape' to L1 via the `.node.defaultChild` property.

L3 Constructs — Patterns (Complete Architectural Solutions)

Level 3 (L3) constructs, also called **Patterns**, represent complete cloud architecture solutions assembled from multiple L2 constructs. A single L3 construct can create dozens of resources — VPC, subnets, ECS cluster, Fargate service, Application Load Balancer, target groups, security groups, IAM roles, CloudWatch log groups — all pre-wired and configured according to AWS best practices. The **aws-ecs-patterns** and **aws-apigateway** modules contain many L3 patterns. Additionally, the **AWS Solutions Constructs** library provides 70+ vetted patterns for common architectural combinations.

Example: `new ecsPatterns.ApplicationLoadBalancedFargateService(this, 'MyService', { taskImageOptions: { image: ecs.ContainerImage.fromRegistry('nginx') } })`

Note: L3 patterns are opinionated. They are great for getting started fast, but you may need to drop to L2 for fine-grained customization.

Escape Hatches — Dropping Between Levels

When a higher-level construct doesn't expose a property you need, CDK provides **escape hatches** — mechanisms to access the underlying lower-level construct and set raw CloudFormation properties directly. This ensures you are never blocked by the framework's abstractions.

```
// Escape hatch: access the L1 CfnBucket from an L2 Bucket
const bucket = new s3.Bucket(this, 'MyBucket');

// Get the underlying L1 CfnBucket
const cfnBucket = bucket.node.defaultChild as s3.CfnBucket;

// Set a CloudFormation property not exposed by L2
cfnBucket.addPropertyOverride('ObjectLockEnabled', true);

// Alternatively – use raw overrides
cfnBucket.cfnOptions.deletionPolicy = cdk.CfnDeletionPolicy.RETAIN;
```

Chapter 4

Stacks, Apps & Environments in Depth

Anatomy of a CDK App

Every CDK application follows the same structural hierarchy. Understanding this tree is essential to reasoning about logical IDs, cross-stack references, and synthesis behavior.

Layer	Class	Represents	Deployment Unit
1 (Root)	App	Entire CDK program; synthesis root	No AWS resource
2	Stack	CloudFormation Stack; account+region scoped	One CF Stack
3	Construct (L3)	Architectural pattern (VPC + ECS + ALB)	Multiple resources
4	Construct (L2)	Single AWS service with helpers	1–5 CF resources
5 (Leaf)	Construct (L1/Cfn*)	Raw CloudFormation resource	Exactly 1 CF resource

Logical ID

Every construct in CDK has a **Logical ID** — a unique string identifier within its parent scope. CDK uses the construct's position in the construct tree to generate CloudFormation Logical IDs (e.g., `MyStackMyBucketABCD1234`). These IDs are critical: CloudFormation uses them to track resource identity across deployments. If a Logical ID changes (because you renamed or moved a construct), CloudFormation treats it as a **delete + recreate** of the resource — which can be catastrophic for stateful resources like RDS databases or S3 buckets with data. Always use stable, explicit IDs for stateful resources.

Example: `new s3.Bucket(this, 'UserDataBucket')` — `'UserDataBucket'` is the construct's id.

Note: Never rename stateful constructs after initial deployment without a migration plan. Use `physicalName` properties to set explicit CloudFormation resource names when needed.

Token

A **Token** is CDK's mechanism for representing values that are not yet known at synthesis time but will be resolved during CloudFormation deployment. For example, the ARN of a new S3 bucket is not known until AWS actually creates it — but you often need to reference it in a Lambda environment variable or IAM policy. CDK represents such values as Tokens (e.g., `${Token[TOKEN.123]}` in synthesized templates). CloudFormation resolves these to actual values during deployment using its **`Fn::GetAtt`** and **`Ref`** intrinsic functions.

Example: `bucket.bucketArn` — this is a Token at synthesis time; CloudFormation resolves it to the real ARN.

Note: You cannot use Token values in JavaScript/Python conditionals at synthesis time. Use CDK's `CfnCondition` for conditional logic that depends on runtime values.

Context

Context is CDK's key-value store for configuration data that is not part of your application logic. Context values can be set in `cdk.json`, passed on the CLI with `--context key=value`, or stored by CDK itself when it performs AWS account lookups (like VPC IDs or AMI IDs). Context allows you to parameterize your CDK app without hardcoding environment-specific values. It also caches the results of expensive AWS API calls (like VPC lookups) so synthesis can be performed offline after the first run.

Example: `const vpId = this.node.tryGetContext('vpId');` — reads a VPC ID from context.

Note: Context values are stored in `cdk.context.json` after lookups. Commit this file to version control to ensure reproducible synthesis.

Chapter 5

The CDK Toolkit & CLI Commands

The **CDK Toolkit** (also called the CDK CLI) is the command-line interface for working with CDK applications. Installed globally via npm, it is the primary tool for synthesizing, deploying, diffing, and destroying CDK stacks. Understanding each command and its flags is essential for daily CDK work.

Installation & Bootstrapping

```
# Install CDK Toolkit globally
$ npm install -g aws-cdk

# Verify installation
$ cdk --version

# Bootstrap your AWS account (required once per account/region)
$ cdk bootstrap aws://ACCOUNT_ID/REGION
# Example:
$ cdk bootstrap aws://111122223333/ap-south-1
```

Bootstrap / CDKToolkit Stack

Bootstrapping is a one-time setup process that provisions the CDK support infrastructure in your AWS account and region. When you run `cdk bootstrap`, CDK creates a CloudFormation Stack called **CDKToolkit** containing: an **S3 Bucket** (to store synthesized assets like Lambda code, Docker images, and large CloudFormation templates), an **ECR Repository** (to store Docker images), and **IAM Roles** (for deployment permissions). Without bootstrapping, CDK cannot deploy stacks that contain assets. Bootstrapping is required once per account-region combination and must be updated when CDK releases new bootstrap versions.

Example: `cdk bootstrap aws://123456789012/us-east-1` — bootstraps the us-east-1 region.

Note: In enterprise setups, the bootstrap IAM roles can be customized for least-privilege access, and the S3 bucket can be encrypted with a customer-managed KMS key.

Core CLI Commands — Reference Table

Command	Purpose	Key Flags	When to Use
<code>cdk init</code>	Scaffold a new CDK project	<code>--language=typescript python java csharp go</code>	Starting a new CDK application
<code>cdk synth</code>	Synthesize CloudFormation template(s)	<code>--output</code> , <code>--quiet</code> , <code>--no-staging</code>	Review what CDK will generate; CI validation
<code>cdk diff</code>	Show what will change vs deployed state	<code>--security-only</code> , <code>--fail</code>	Before every deploy — review changes safely

Command	Purpose	Key Flags	When to Use
cdk deploy	Deploy stack(s) to AWS	--require-approval, --outputs-file, --hotswap	Apply infrastructure changes
cdk destroy	Delete stack(s) from AWS	--force, --all	Tear down environments (dev/test cleanup)
cdk list (ls)	List all stacks in the app	--long	See all stacks and their environments
cdk watch	Deploy on code change (dev mode)	--hotswap-fallback	Fast iteration in development
cdk doctor	Check CDK installation health	—	Troubleshoot CDK setup issues
cdk context	Manage context cache	--reset, --clear	Refresh cached VPC/AMI lookups
cdk import	Import existing AWS resources into a stack	--record-resource-mapping	Migrate existing infra to CDK management

Hotswap Deployment

Hotswap is a CDK deploy mode (introduced in CDK v1.122) that skips CloudFormation for certain change types and directly updates the AWS resource instead. For example, changing a Lambda function's code normally takes 1-3 minutes through CloudFormation. With hotswap, CDK detects that only the Lambda code changed, and directly calls `UpdateFunctionCode` — completing in 5-10 seconds. Hotswap supports Lambda code, ECS container images, Step Functions definitions, and CodeBuild project sources. **Hotswap should never be used in production** as it creates drift between the CloudFormation state and actual resource state.

Example: `cdk deploy --hotswap` — fast Lambda code updates in development.

Note: Use `cdk watch` (which uses hotswap internally) for the best iterative development experience. Changes to non-hotswappable resources fall back to CloudFormation.

Chapter 6

Assets, Bundling & Dependencies

Assets

Assets are local files, directories, or Docker images that CDK packages and uploads to AWS as part of the deployment process. They allow you to co-locate your application code (Lambda functions, ECS container definitions, static website files) with your infrastructure code in the same CDK project. There are two types of assets: **(1) File Assets** — local files or directories uploaded to the CDK S3 bucket in the bootstrap stack. Used for Lambda code, configuration files, and website bundles. **(2) Docker Image Assets** — Docker images built locally and pushed to the ECR repository in the bootstrap stack. Used for ECS tasks, Lambda container images. CDK automatically handles asset fingerprinting (using content hash), uploading, and referencing — you don't need to manually manage S3 uploads or ECR pushes.

Example: `lambda.Function.fromAsset('./lambda-code')` — packages the local directory as a Lambda asset.

Note: Assets are uploaded once per unique content hash. If the file hasn't changed, CDK reuses the previously uploaded version, making deployments faster.

Bundling

Bundling is the process of transforming your source code before packaging it as an asset. This is commonly used to: compile TypeScript Lambda functions to JavaScript, install Python dependencies into a Lambda package, minify and bundle web frontend code, or run language-specific build steps. CDK performs bundling inside a Docker container that matches the Lambda/ECS runtime environment, ensuring the bundled artifact is compatible with the target AWS runtime. The **NodejsFunction** and **PythonFunction** L2 constructs handle bundling automatically using esbuild and pip respectively.

Example: `new lambda.NodejsFunction(this, 'Handler', { entry: 'src/handler.ts' })` — TypeScript is compiled automatically.

Note: Bundling requires Docker to be installed locally unless you use CDK's local bundling option with a custom `ILocalBundling` implementation.

Aspects

Aspects are CDK's implementation of the Visitor design pattern applied to the construct tree. An Aspect is a class with a `visit(node)` method that CDK calls on every construct in the tree during synthesis. Aspects are used to apply cross-cutting concerns at scale — for example: adding a specific tag to all resources, enforcing that all S3 buckets have encryption enabled (and throwing a synthesis error if not), or automatically enabling termination protection on all production stacks. Aspects run after the entire construct tree is built, making them ideal for governance and compliance enforcement.

Example: `Aspects.of(app).add(new cdk.Tag('CostCenter', 'Platform-Eng'))`

Note: `cdk-nag` is a popular open-source library built entirely on Aspects. It visits every construct and validates it against AWS security best practices.

CfnOutput

A **CfnOutput** is a CloudFormation Stack Output — a named value exported from a Stack after deployment. Outputs are visible in the CloudFormation console and can be referenced by other Stacks (cross-stack references) or used in CI/CD pipelines via the `--outputs-file` flag of `cdk deploy`. Common uses: exporting a Load Balancer URL, an S3 bucket name, a Lambda ARN, or an API Gateway endpoint so that application configuration scripts can consume them automatically.

Example: `new cdk.CfnOutput(this, 'BucketName', { value: bucket.bucketName, exportName: 'MyBucketName' });`

Note: Cross-stack references via `CfnOutput` create hard dependencies between stacks. Consider SSM Parameter Store for looser coupling between stacks.

Chapter 7

CDK Workflow, Lifecycle & Advanced Terminology

The Complete CDK Synthesis Lifecycle

Understanding how CDK processes your code — from source to deployed AWS resources — helps you reason about synthesis errors, deployment failures, and timing issues.

- 1. Construction Phase:** CDK executes your code top-to-bottom. Constructs are instantiated and organized into the construct tree. No AWS calls are made yet.
- 2. Preparation Phase:** CDK walks the construct tree and calls `prepare()` on every node. Cross-references between stacks are resolved into CloudFormation Exports/Imports at this stage.
- 3. Validation Phase:** CDK calls `validate()` on every node. Constructs can throw errors here to prevent invalid configurations from being synthesized. Aspects also run during this phase.
- 4. Synthesis Phase:** CDK calls `synthesize()` on every construct in depth-first order, generating the CloudFormation template JSON and the assets manifest in the `cdk.out` directory.
- 5. Deployment Phase:** The CDK Toolkit reads the synthesized output from `cdk.out`, uploads assets to S3/ECR, and calls the CloudFormation APIs to create a Change Set and execute the deployment.
- 6. Post-Deployment:** CloudFormation provisions/updates all resources, resolves Tokens to actual values, and reports success or failure. CDK Outputs are printed.

JSII — Multi-Language Support

JSII (JavaScript Interop Interface) is the technology that enables CDK to support multiple programming languages from a single TypeScript codebase. CDK is authored in TypeScript, and JSII compiles it into language-specific bindings for Python, Java, C#, and Go. When you use the Python CDK library (`aws-cdk-lib` on PyPI), you are interacting with a JSII-generated Python wrapper that delegates all calls to a bundled Node.js subprocess running the original TypeScript code. This is why Node.js must be installed even for Python, Java, or C# CDK applications.

Example: `import aws_cdk as cdk # Python — JSII-generated bindings for aws-cdk-lib`

Note: JSII imposes some constraints on CDK construct API design (e.g., no method overloading), but these rarely affect end users of CDK libraries.

CDK Pipelines

CDK Pipelines is a high-level L2 construct library built on top of AWS CodePipeline that creates **self-mutating CI/CD pipelines** for CDK applications. 'Self-mutating' means that when you push changes to your CDK pipeline definition, the pipeline first updates itself to match the new definition, then deploys the application. This enables true GitOps: the pipeline definition is code, and changes to the pipeline flow through the pipeline itself. CDK Pipelines natively supports multi-account, multi-stage deployments (`dev` → `staging` → `prod`) with automated testing stages between environments, manual approval gates, and cross-account deployment roles.

Example: `new CodePipeline(this, 'Pipeline', { synth: ShellStep('Synth', { ... }) })`

Note: CDK Pipelines require the bootstrap stack to include deployment roles. Use `cdk bootstrap --trust PIPELINE_ACCOUNT_ID` for cross-account scenarios.

Nested Stacks

Nested Stacks are Stacks created inside another Stack (the parent). They map to CloudFormation Nested Stacks (AWS::CloudFormation::Stack resource type). Nested Stacks are primarily used to overcome CloudFormation's 500-resource-per-stack limit by splitting a large stack into smaller child stacks that are deployed together as one unit. Unlike top-level Stacks (which are independently deployable), Nested Stacks are always deployed as part of their parent Stack. They share the parent's deployment lifecycle and cannot be deployed independently.

Example: `const nested = new NestedStack(this, 'DatabaseLayer');` — creates a nested stack for database resources.

Note: Prefer splitting into independent top-level Stacks over Nested Stacks when possible. Nested Stack updates are slower because CloudFormation must upload the child template to S3.

RemovalPolicy

The **RemovalPolicy** controls what happens to an AWS resource when its CDK construct or Stack is deleted. This is critical for stateful resources. CDK provides three options: **DESTROY** — the resource is deleted when the stack is deleted (default for most resources, dangerous for databases); **RETAIN** — the resource is kept in AWS even after the Stack is deleted (safe for production databases and S3 buckets with data); **SNAPSHOT** — a snapshot is taken before deletion (supported by RDS, ElastiCache, Neptune, DocumentDB). Setting the correct RemovalPolicy is one of the most important production-readiness steps in any CDK application.

Example: `new s3.Bucket(this, 'DataBucket', { removalPolicy: cdk.RemovalPolicy.RETAIN })`

Note: In development, DESTROY is convenient. In production, always use RETAIN or SNAPSHOT for any resource containing important data.

Quick Reference — All Key CDK Terms

Term	One-Line Definition
App	Root construct; entry point of every CDK program; owns all Stacks
Stack	Unit of deployment; maps to one CloudFormation Stack
Construct	Fundamental building block; every CDK component is a Construct
L1 (Cfn*)	1:1 CloudFormation resource; maximum control, maximum verbosity
L2	Curated AWS construct with sensible defaults and helper methods
L3 / Pattern	Pre-built multi-resource architectural solution

Term	One-Line Definition
Environment	AWS Account + Region combination for Stack deployment
Logical ID	Unique CloudFormation identifier auto-generated from construct path
Token	Placeholder for runtime values not known at synthesis time
Context	Key-value configuration store; parameterizes CDK apps
Asset	Local file or Docker image uploaded to S3/ECR during deploy
Bundling	Pre-deployment transformation of source code (compile/install deps)
Aspect	Visitor pattern applied to all constructs; used for governance
CfnOutput	CloudFormation Stack Output; exports values to other stacks/scripts
Bootstrap	One-time per-account setup creating CDKToolkit support resources
Hotswap	Fast direct-update deployment for dev; bypasses CloudFormation
JSII	Technology enabling CDK's multi-language support from TypeScript
CDK Pipelines	Self-mutating CI/CD pipeline construct built on CodePipeline
Nested Stack	Stack-within-a-Stack; used to overcome 500-resource CF limit
RemovalPolicy	Controls resource fate on Stack deletion: DESTROY/RETAIN/SNAPSHOT
cdk.out	Synthesis output directory containing CF templates + assets manifest
Synthesis	The process of converting CDK code into CloudFormation templates
Escape Hatch	Mechanism to access L1 from L2; used for unsupported properties
Construct Hub	Public registry of open-source CDK construct libraries

■ Next Steps: Visit cdkworkshop.com for a free hands-on tutorial, constructs.dev (Construct Hub) to explore community libraries, and docs.aws.amazon.com/cdk/api/v2 for the complete API reference. The CDK GitHub repository (github.com/aws/aws-cdk) is also an excellent learning resource — reading the L2 construct source code shows exactly how AWS recommends configuring each service.